



안양대학교 일반대학원 · 미세먼지관리전공

BRANCH & PULL REQUEST

교수님과 스마트하게 협업하는 방법

Session 2 — 브랜치 전략과 GitHub 협업 완전 정복

전제 Session 1 완료 (저장소 생성)

목표 교수님께 코드 리뷰 받기

시간 2시간 (Zoom)

SESSION 1 복습

지난 세션 확인 & 오늘의 새로운 문제



Session 1 에서 배운 것

- ▶ Git 설치 & 초기 설정
- ▶ add → commit → push
- ▶ GitHub 저장소 연동
- ▶ .gitignore 설정
- ▶ 일일 Git 루틴



아직 해결 안 된 문제들

- ▶ 실험적 분석 → main 코드가 망가질까 두려움
- ▶ 교수님께 코드 리뷰 어떻게 받나?
- ▶ 두 분석 방향을 동시에 시도하고 싶다
- ▶ 최종본과 실험본을 어떻게 분리?
- ▶ 팀원이 동시에 수정하면 어떻게?

💡 모든 문제의 해답: 브랜치(Branch) + Pull Request

Session 1 저장소에서 그대로 이어갑니다

오늘 실습은 새 프로젝트가 아니라, 지난 시간에 만든 **pm25-thesis** 저장소에서 이어집니다.

지난 시간에 만든 연구 저장소로 이동

```
cd ~/pm25-thesis
```

main 브랜치와 GitHub 연결 상태 확인

```
git checkout main
```

```
git pull origin main
```

```
git remote -v
```

오늘 사용할 파일 확인

```
ls analysis
```

```
pm25_analysis.py
```

copy

analysis/pm25_analysis.py에 기존 코드가 없다면 main에서 먼저 추가

```
pm25_threshold = 25 # 국내 PM2.5 일평균 관리 기준 (실습용)
```

```
def is_high_pm25(value):
```

```
    return value ≥ pm25_threshold
```

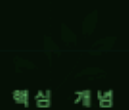
기존 코드를 새로 추가했다면 main에 먼저 저장

```
git add analysis/pm25_analysis.py
```

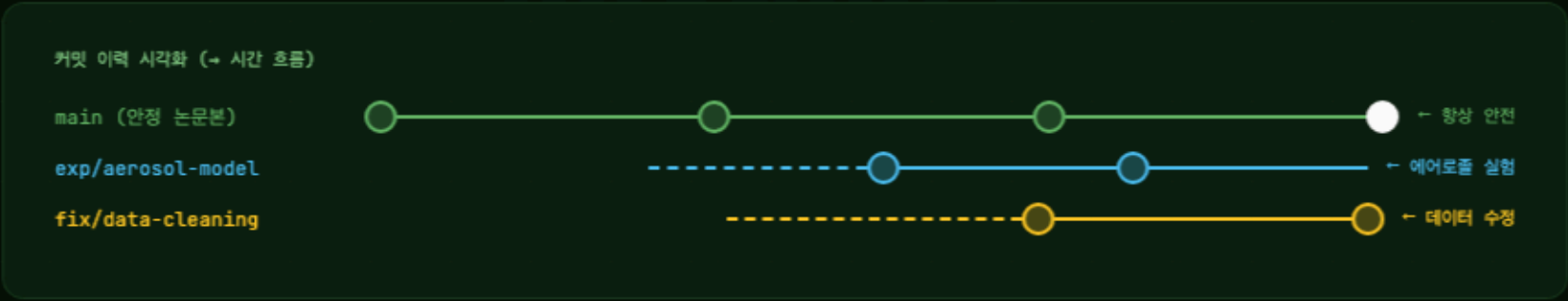
```
git commit -m "PM2.5 기준값 함수 추가"
```

```
git push
```

copy



브랜치란? — 평행 연구 우주



🌿 **브랜치의 핵심:** 복사본이 아닙니다. main과 독립된 작업 공간입니다. 실험이 실패해도 main은 절대 안전합니다.

본 슬라이드는 `main` 브랜치로 자동 생성되었습니다. 브랜치 이름은 `exp/`로 시작하는 것이 좋습니다.

브랜치 만들고 PM2.5 기준값 실험하기

```
# 항상 main 최신 상태에서 출발

git checkout main
git pull origin main


# 현재 브랜치 확인

git branch
* main


# 새 브랜치 생성 + 이동 (한 번에)
git checkout -b experiment/pm25-threshold
Switched to a new branch 'experiment/pm25-threshold'


# analysis/pm25_analysis.py에서 같은 줄을 수정
pm25_threshold = 35 # WHO 기준 적용 실험


# 이 브랜치에서 수정 내용 커밋
git add analysis/pm25_analysis.py
git commit -m "PM2.5 기준값 실험: WHO 기준 적용"


# 브랜치를 GitHub에 업로드
git push -u origin experiment/pm25-threshold


# 다시 main으로 돌아가기
git checkout main
```

copy

브랜치 전략

미세먼지 연구에 맞는 브랜치 명명 규칙

experiment/ — 새 분석 시도

- ▶ experiment/pm25-threshold-test
- ▶ experiment/ml-pm25-prediction
- ▶ experiment/aermod-sigma-param
- ▶ experiment/seasonal-source-apportionment

fix/ — 오류 수정

- ▶ fix/airkorea-data-cleaning
- ▶ fix/pm25-threshold-correction
- ▶ fix/datetime-format-error
- ▶ fix/outlier-removal-logic

docs/ — 논문 작성

- ▶ docs/chapter2-methods
- ▶ docs/chapter3-results-draft
- ▶ docs/abstract-revision
- ▶ docs/figure-captions

feature/ — 새 기능 추가

- ▶ feature/interactive-map-viz
- ▶ feature/auto-report-generator
- ▶ feature/realtime-sensor-api
- ▶ feature/cloud-data-pipeline

핵심 개념

Pull Request — 교수님께 코드 리뷰 요청

- 1

브랜치에서 작업 완료 후 GitHub에 push

experiment/pm25-threshold 브랜치에서 PM2.5 기준값 실험을 커밋하고 push
- 2

GitHub 웹에서 Pull Request 생성

GitHub → 저장소 → "Compare & pull request" 노란 버튼 → 제목과 설명 작성 → 정광용 교수님을 Reviewer로 지정
- 3

교수님이 코드 한 줄 한 줄 리뷰

변경된 코드 라인마다 댓글 가능 — "이 부분 PM2.5 보정 계수 확인해보세요", "이 함수는 더 효율적으로 작성 가능"
- 4

수정 반영 후 교수님 Approve → Merge

피드백 수정 → 재 push → 교수님 승인 → main에 병합. 검증된 코드만 main으로 들어옵니다.

교수님이 좋아하는 PR 작성법



좋은 PR 제목

- ▶ [분석] PM2.5 계절별 농도 분포 함수 완성
- ▶ [수정] AirKorea -999 결측값 처리 개선
- ▶ [실험] AERMOD 수평확산 σ 파라미터 조정
- ▶ [문서] 3장 결과 및 고찰 초안
- ▶ [AI] Random Forest 예측 모델 v1



PR 본문 템플릿

- ▶ ## 변경 내용 (무엇을 했는가)
- ▶ - 계절별 PM2.5 평균 계산 함수 추가
- ▶ ## 확인 요청 사항
- ▶ - 계절 구분 기준이 맞는지 확인 부탁드립니다
- ▶ ## 관련 수업 / 이슈
- ▶ - 파이썬 통계분석 수업 3주차 연계

```
# GitHub CLI 로 PR 생성 (고급)
gh pr create \
  --title "[분석] PM2.5 계절별 농도 분포 함수 완성" \
  --reviewer [교수 GitHub ID] \
  --base main \
  --body "계절별 PM2.5 평균 계산 완료. 계절 구분 기준 확인 부탁드립니다."
```

copy

병합 전략

Clean Merge — 브랜치를 main에 합치기

방법 1: GitHub 웹에서 (권장) — Merge pull request 버튼 클릭

copy

방법 2: 터미널에서 직접 병합

① main으로 이동

`git checkout main`

② 방금 만든 실험 브랜치 병합

`git merge experiment/pm25-threshold`

Updating a3f9c21..d74b2f9

Fast-forward

analysis/pm25_analysis.py | 2 +-
1 file changed, 2 insertions(+), 0 deletions(-)

③ GitHub에 push

`git push`

④ 완료된 브랜치 삭제 (선택)

`git branch -d experiment/pm25-threshold`

`git push origin --delete experiment/pm25-threshold`

💡 PR을 통해 Merge하면 **리뷰 기록이 영구 보존**됩니다. 논문 방법론 설명 시 "코드 리뷰 이후 수정" 근거 자료로 활용 가능

Conflict를 일부러 만들어보기

충돌은 “같은 파일의 같은 줄”을 서로 다르게 바꿨을 때 생깁니다. 오늘은 `pm25_threshold` 한 줄로 안전하게 재현합니다.

```
# ① main 최신 상태에서 충돌 실습 브랜치 생성
git checkout main
git pull origin main
git checkout -b conflict/who-threshold-note

# ② 브랜치에서 pm25_threshold 줄을 이렇게 수정 후 커밋
pm25_threshold = 35 # WHO 권고 기준
git add analysis/pm25_analysis.py
git commit -m "WHO 기준값 설명 추가"

# ③ main으로 돌아와 같은 줄을 다르게 수정 후 커밋
git checkout main
pm25_threshold = 25 # 국내 환경기준 유지
git add analysis/pm25_analysis.py
git commit -m "국내 기준값 설명 추가"

# ④ 이제 병합하면 충돌 발생
git merge conflict/who-threshold-note
```

copy

문 제 해 결

충돌(Conflict) — 발생 원리와 해결

⚠ **충돌이란?** 두 브랜치가 같은 파일의 같은 줄을 다르게 수정했을 때 Git이 자동 병합 불가 → 수동 해결 필요

```
# 충돌 발생 시 파일 내부에 이런 표시가 생김
<<<<<<< HEAD (현재 브랜치)
pm25_threshold = 25 # 국내 환경기준 유지
=====
pm25_threshold = 35 # WHO 권고 기준
>>>>>>> conflict/who-threshold-note

# 해결: 파일을 직접 편집해서 원하는 버전만 남기고 표시 삭제
pm25_threshold = 35 # WHO 권고 기준 (수업 실습용)

# 충돌 해결 후 커밋
git add analysis/pm25_analysis.py
git commit -m "충돌 해결: PM2.5 기준값 WHO 기준 적용"
git push

# 충돌이 두렵다면 → 병합 취소
git merge --abort
```

copy

프로젝트 관리

GitHub Issues — 연구 할 일 관리



버그 리포트

분석 코드 오류 발견 시 Issue 등록 → 수정 PR
연결 → 해결 시 Close. 오류 이력 완전 추적



연구 아이디어

새 분석 방향 제안 → Issue 댓글로 교수님과 논
의 → 결정 후 브랜치 생성. 아이디어도 기록!



논문 진행 관리

서론 작성, 방법론 작성 등 섹션별 Issue → 완료
시 Close. 논문 진행도 한눈에 파악

실용 Issue 라벨 (미세먼지 연구용):

데이터수집

모델링

버그

논문작성

core-paper

screening

교수님검토요청

완료



논문 스크리닝에도 Issue 사용: 논문 1편 = Issue 1개로 등록하고, 제목·DOI/URL·초록·읽을 이유·A/B/C 판단을 남깁니다. 핵심 논문은

core-paper

, 제외 논문은

exclude

로 표시합니다.

실제 시나리오

교수님과의 주간 협업 루틴

월

주간 연구 계획 — GitHub Issue 등록

"이번 주: AirKorea 2023년 PM2.5 데이터 정제 및 계절 평균 계산" Issue 등록 → 브랜치 생성

화-목

브랜치에서 집중 작업 & 매일 커밋

feature/data-cleaning 브랜치에서 작업 → 매일 저녁 의미있는 커밋 + push

금

PR 생성 → 교수님께 리뷰 요청

주간 작업 완료 → PR 작성 (변경 내용 + 확인 요청 사항) → 교수님 Reviewer 지정

다음 주

피드백 반영 → Approve → Merge

교수님 코멘트 수정 → 재 push → Approve → main 병합 → 새 주간 계획 시작

외부 코드 활용

Fork & Clone — 외부 코드 가져오기



Fork (포크)

- ▶ 다른 사람 저장소를 내 계정으로 복사
- ▶ 원본에 영향 없이 자유롭게 수정
- ▶ 기상청, AirKorea 공개 코드 활용 시
- ▶ GitHub 웹에서 버튼 한 번으로 완료
- ▶ Pull Request로 원본에 기여도 가능



Clone (클론)

- ▶ GitHub 저장소를 로컬로 다운로드
- ▶ 교수님 저장소 클론해서 실습
- ▶ 연구실 공용 코드 받아오기
- ▶ `git clone [URL]` 명령 한 번
- ▶ 자동으로 origin 원격 등록됨

저장소 클론 (로컬에 다운로드)

```
git clone https://github.com/kwangorithm/finddust.git
cd finddust
```

최신 상태로 업데이트

```
git pull origin main
```

포크한 저장소에서 원본 최신화

```
git remote add upstream https://github.com/원본저장소.git
git fetch upstream
git merge upstream/main
```

copy

유용한 기능

git stash — 작업 임시 저장

💡 **언제 쓰나?** 브랜치 A에서 작업 중인데 갑자기 브랜치 B로 이동해야 할 때. 커밋하기엔 아직 미완성 상태일 때.

```
# 시나리오: experiment 브랜치에서 분석 중인데
# 교수님이 긴급하게 main 코드 수정 요청

# ① 현재 작업 임시 저장
git stash save "PM2.5 계절분석 작업 중 - 미완성"
Saved working directory and index state: PM2.5 계절분석 작업 중 - 미완성

# ② main으로 이동해서 작업
git checkout main
# ... 긴급 수정 완료 ...

# ③ 원래 브랜치로 돌아와 stash 복원
git checkout experiment/seasonal-analysis
git stash pop
# 임시 저장한 작업이 그대로 복원됨

# stash 목록 보기
git stash list
```

copy

전공 연계

브랜치 전략 — 전공 수업별 적용



대기확산모델링 이론

- ▶ main: 최종 검증된 AERMOD 설정
- ▶ experiment/aermod-sigma: 확산계수 실험
- ▶ experiment/calpuff-compare: 모델 비교
- ▶ 실패한 실험도 보존 → 논문 방법론 근거



AI기반 미세먼지 예측 및 관리

- ▶ feature/lstm-model: LSTM 모델 시도
- ▶ feature/random-forest: RF 모델 비교
- ▶ feature/xgboost: XGBoost 실험
- ▶ PR로 세 모델 성능 비교 토론



사례 연구를 통한 논문 작성 방법론

- ▶ docs/ch1-introduction: 1장 초안
- ▶ docs/ch2-methods: 2장 연구방법
- ▶ docs/ch3-results: 3장 결과
- ▶ PR = 교수님 논문 피드백 창구



비용편익 분석실습

- ▶ feature/cost-benefit-calc: 계산 모듈
- ▶ experiment/scenario-a, b, c: 시나리오별
- ▶ 각 정책 시나리오가 독립 브랜치
- ▶ 비교 후 최적안 main에 Merge

자동화 맛보기

GitHub Actions — push 하면 자동으로!



자동 코드 검사

push 할 때마다 Python 문법 오류, 코드 스타일 자동 검사. 오류 있으면 PR에 경고 표시



자동 그래프 생성

데이터 업데이트되면 PM2.5 시계열 그래프 자동 재생성. figures/ 폴더 자동 업데이트



자동 알림

PR 생성 시 GitHub 알림으로 리뷰 요청 전달. 매일 정해진 시간에 코드 자동 실행

💡 Actions는 Session 3-4에서 Claude Code, Codex와 연계해 더 강력하게 활용할 예정입니다. 지금은 "자동화할 수 있구나"만 이해해도 충분합니다.

치트시트

Session 2 명령어 치트시트

===== 브랜치 관리 =====

git branch # 브랜치 목록

git branch -a # 원격 브랜치 포함 전체

git checkout -b 브랜치명 # 생성 + 이동

git checkout main # main으로 이동

git branch -d 브랜치명 # 브랜치 삭제

===== 병합 =====

git merge 브랜치명 # 현재 브랜치에 병합

git merge --abort # 충돌 시 병합 취소

===== 원격 브랜치 =====

git push -u origin 브랜치명 # 브랜치 GitHub push

git pull origin 브랜치명 # 원격 브랜치 pull

git fetch --all # 모든 원격 정보 갱신

===== stash =====

git stash save "설명" # 임시 저장

git stash pop # 복원

git stash list # 목록

copy

세션 요약

Session 2 핵심 정리



브랜치 핵심

- ▶ 실험은 항상 브랜치에서
- ▶ main = 항상 안정본
- ▶ checkout -b 브랜치 생성
- ▶ 명명 규칙 준수
- ▶ push -u origin 브랜치명



PR 핵심

- ▶ GitHub 웹에서 PR 생성
- ▶ 교수님을 Reviewer로
- ▶ 제목: [유형] 내용 형식
- ▶ 본문: 변경 + 확인 요청
- ▶ Approve 후 Merge



다음 세션 예고

- ▶ Claude Code 설치
- ▶ AI와 PM2.5 코드 작성
- ▶ 프롬프트 전략
- ▶ 오류 AI 디버깅
- ▶ Git + AI 연동

과제

Session 2 과제



다음 세션 전까지 완료

1. **experiment/season-analysis** 브랜치 생성 후 작업

봄/여름/가을/겨울 PM2.5 평균 계산 Python 코드 작성 (샘플 데이터로도 OK)

2. 의미있는 커밋 3개 이상 (각 기능 완성 시마다)

예: "봄 필터링", "계절 평균 계산", "박스플롯 추가"

3. GitHub에 **Pull Request** 생성

제목: [분석] 계절별 PM2.5 평균 초안 | 본문에 분석 방법 간략 설명 + 확인 요청 1가지

4. GitHub Issues에 **연구 계획 4가지 이상** 등록

예: "AirKorea 데이터 수집", "AERMOD 설치", "2장 연구방법 초안", "기상 데이터 연동"

5. Google Scholar에서 찾은 논문 3편을 **Issue**로 등록하고 A/B/C 스크리닝

제목, DOI/URL, 초록, 왜 읽어야 하는지, 내 논문에서 쓸 위치를 기록

6. **fix/ 브랜치** 하나 추가 실습 (Session 1 코드에서 수정할 것 찾기)

변수명 개선, 주석 추가, 코드 리팩터링 등 무엇이든

7. 🎁 **보너스**: GitHub Insights 캡처 → 본인 커밋 활동 현황 카카오톡 방에 공유